90-DAY JOURNEY

DAY 04/90

AI SECURITY & RED TEAMING

Day 4 — HTTP & REST APIs for AI Applications

Level: Beginner | Study time: 3 hours | 3 parts + hands-on lab

Phase 1 · Foundation · For students, working professionals & freelancers

IN THIS GUIDE

PART 1 HTTP Fundamentals — The Language of the Web

PART 2 Headers, REST Design & How AI Apps Use APIs

PART 3 API Security Thinking & Defenses

PART 4 Hands-on Practical Worksheet

OBJECTIVE

Understand exactly what travels between a browser, an AI app backend and an LLM provider: HTTP requests, responses, methods, status codes, headers and REST design — then build and test your own API with curl.

Educational use only — test only with authorization

PART 1

HTTP Fundamentals — The Language of the Web

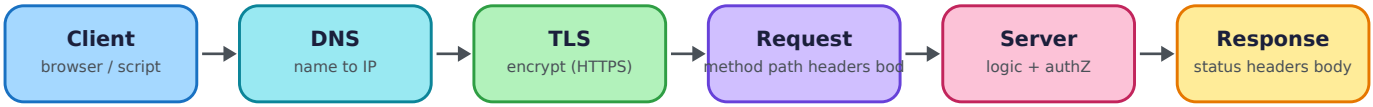
Hour	Focus	What you do today
HR 1	Learn	HTTP anatomy, methods, status codes, headers (Part 1) and REST + AI API patterns (Part 2).
HR 2	Build	Build a small FastAPI chat-history API and explore it with curl and `/docs` (Part 4, Tasks 1–4).
HR 3	Red Team	Find the 3 planted API weaknesses in your own API, fix them, retest (Tasks 5–6). Last 10 min: write notes in `DAY-XX.md`.

1. Why HTTP matters for AI red teaming

Every AI application you test talks over HTTP: the chat UI calls the backend API, the backend calls the LLM provider API, a RAG system calls a vector database API, and agents call tool APIs. If you can read and modify HTTP traffic, you can see what the AI app **really** does — not just what the UI shows.

> ▸ **EXAMPLE**
>
> A chatbot UI hides the 'model' and 'temperature' settings. Looking at the HTTP request, you notice the browser sends `"model": "basic"` and the full system prompt. That tells you the client controls settings that should be decided by the server — a design weakness worth testing.

2. Anatomy of a request and a response




```
● raw HTTP request + response

POST /api/chat?lang=en HTTP/1.1          <- method, path, query string, version
Host: chat.example.com                   <- headers start
Authorization: Bearer eyJhbGciOi...
Content-Type: application/json
Cookie: session=8f2c...
                                         <- empty line, then body
{"message": "Explain HTTP in simple words"}

HTTP/1.1 200 OK                          <- status line
Content-Type: application/json
Set-Cookie: session=8f2c...; HttpOnly; Secure

{"reply": "HTTP is how a browser and a server talk..."}
```

3. HTTP methods

Method	Purpose	Security note
`GET`	Read a resource.	Should never change data. Parameters appear in URLs and logs — no secrets in query strings.
`POST`	Create / submit (chat message, upload).	Most AI endpoints use POST. Body is where injection tests go.
`PUT` / `PATCH`	Replace / partly update a resource.	Check whether you can update fields or objects you do not own.

Method	Purpose	Security note
DELETE	Remove a resource.	Is ownership checked? Can user B delete user A's conversation?
OPTIONS	Ask which methods are allowed (CORS preflight).	Reveals allowed methods / origins — useful recon.
HEAD	Like GET, headers only.	Quick checks without downloading the body.

4. Status codes — what the server is telling you

Class	Common codes	Meaning for a tester
2xx **Success**	200 OK, 201 Created, 204 No Content	It worked — check if it should have worked for this user.
3xx **Redirect**	301, 302, 304	Login redirects, open-redirect checks later.
4xx **Client error**	400, 401, 403, 404, 405, 422, 429	Controls and validation are working (or leaking info via different messages).
5xx **Server error**	500, 502, 503, 504	Unhandled input or a crashed upstream (e.g. LLM provider timeout) — investigate.

WHO ARE YOU? — HOW TODAY HELPS YOU

● STUDENT
HTTP is asked in every security interview. Learn the request/response anatomy by heart — draw it from memory tomorrow morning.

● WORKING PRO
Open DevTools (F12 → Network) on the AI tools you use at work and just read the requests. You will learn a lot in 10 minutes.

● FREELANCER
Clients often give you only a URL and API docs. Reading HTTP fast lets you scope and price API engagements accurately.

PART 2

Headers, REST Design & How AI Apps Use APIs

1. Headers every red teamer must know

Header	What it does	Why you care
`Authorization`	Carries a token / API key (`Bearer ...`).	Who the server thinks you are. Missing / weak checks = auth bugs.
`Cookie` / `Set-Cookie`	Session handling.	Flags `HttpOnly`, `Secure`, `SameSite` protect sessions.
`Content-Type`	Format of the body (`application/json`).	Changing it can reveal different parsers and validation gaps.
`Origin` + `Access-Control-Allow-*`	CORS — which websites may call the API from a browser.	`*` with credentials or reflected origins = misconfiguration.
`X-Forwarded-For`	Client IP when behind a proxy.	If trusted blindly, rate limits based on IP can be bypassed.
`Server`, `X-Powered-By`	Software / version info.	Recon: framework and version hints.
`Retry-After`, `X-RateLimit-*`	Rate limit info.	Shows limits exist — important for LLM cost-abuse testing (Day 20, 23).

2. REST in plain words

REST is a style of designing APIs around **resources** (nouns) identified by URLs, acted on with HTTP **methods** (verbs), usually exchanging **JSON**. Each request carries everything needed (stateless).

Action	Method + endpoint	Typical AI app example
List	`GET /api/conversations`	Show my chat history sidebar.
Create	`POST /api/conversations`	Start a new chat.
Read one	`GET /api/conversations/42`	Open chat number 42 — whose chat is 42?
Update	`PATCH /api/conversations/42`	Rename the chat.
Delete	`DELETE /api/conversations/42`	Delete the chat.
Action	`POST /api/conversations/42/messages`	Send a message to the model.
Upload	`POST /api/documents`	Upload a PDF for RAG — untrusted content enters here.

> **! REMEMBER**
> IDs in URLs (`/conversations/42`) are a classic test point. The server must check that **the logged-in user owns object 42** on every request. You will test this properly on Day 6 and Day 22.

3. What an LLM provider API request looks like

Most LLM APIs follow a similar shape: a model name, a list of messages with roles, and generation settings. Exact field names differ by provider — always read the provider's docs.

● typical LLM API request (generic)

```
POST https://llm-provider.example/v1/chat        (shape varies by provider)
Authorization: Bearer <server-side API key - never in the browser>
Content-Type: application/json

{
  "model": "some-model-name",
  "messages": [
    {"role": "system", "content": "You are a helpful college assistant."},
    {"role": "user",   "content": "When is the library open?"}
  ],
  "temperature": 0.2,
  "max_tokens": 300,
  "stream": false
}
```

Field	Security question
API key	Is it only on the server? A key in browser JS or a mobile app can be extracted and abused.
`system` **message**	Is it added by the backend, or sent from the client (user-editable)?
`model`, `max_tokens`	Can the user pick an expensive model or huge outputs? (cost abuse — Day 20/23)
`stream: true`	Streaming uses Server-Sent Events (SSE). Output filters must work on streamed chunks too.

4. curl cheat sheet

● curl essentials

```
curl -i  http://127.0.0.1:8000/health                       # -i show response headers
curl -v  http://127.0.0.1:8000/health                       # -v full request + response
curl -i -X POST http://127.0.0.1:8000/api/conversations \
     -H "Content-Type: application/json" -d '{"title":"exam prep"}'
curl -i -X DELETE http://127.0.0.1:8000/api/conversations/1
curl -i -X OPTIONS http://127.0.0.1:8000/api/conversations
curl -s http://127.0.0.1:8000/openapi.json | jq '.paths | keys'   # list endpoints
```

! SCOPE RULE

Test only systems you own or have written permission to test: your own laptop, your own lab VM, intentionally vulnerable practice labs (PortSwigger, OverTheWire, Gandalf) or a client engagement with a signed scope.

PART 3

API Security Thinking & Defenses

1. OWASP API Security Top 10 (2023) — the API tester's map

ID	Risk	How it shows up in AI apps
API1	Broken Object Level Authorization (BOLA)	Reading another user's conversation or uploaded document by changing an ID.
API2	Broken Authentication	Weak tokens, API keys that never expire, keys exposed in the frontend.
API3	Broken Object Property Level Authorization	Setting `role`, `owner`, `plan` or `model` fields you should not control (mass assignment).
API4	Unrestricted Resource Consumption	No limits on message size, tokens or requests → huge LLM bills, slow service.
API5	Broken Function Level Authorization	Normal user calling `/api/admin/...` endpoints (e.g. change system prompt).
API6	Unrestricted Access to Sensitive Business Flows	Automating free-trial signups to get free LLM credits.
API7	Server-Side Request Forgery (SSRF)	'Summarize this URL' feature fetching internal addresses.
API8	Security Misconfiguration	Debug mode, verbose errors, permissive CORS, exposed docs in production.
API9	Improper Inventory Management	Old `/v1/` endpoint without the new guardrails still online.
API10	Unsafe Consumption of APIs	Trusting responses from third-party APIs / tools without validation.

! KEY POINT

LLM-specific risks (OWASP LLM Top 10) sit **on top of** normal API risks. Many real AI app findings are classic API bugs — BOLA, mass assignment, missing rate limits — not clever prompts.

2. API recon — questions to ask in an authorized test

01 Is there API documentation (`/docs`, `/openapi.json`, `/swagger`)? Which endpoints exist?

02 Which endpoints need authentication — and which do not?

03 Which IDs appear in URLs or bodies? Are they guessable (1, 2, 3...)?

04 Which fields does the API accept? Does it accept **extra** fields you were not supposed to send?

05 What happens with wrong types, empty values, very long values? (status code + error message)

06 Are there size limits and rate limits on chat / upload endpoints?

07 Do error messages reveal stack traces, file paths, model names or internal hosts?

3. Defense thinking

Weakness	Defense
Mass assignment	Use explicit input schemas (e.g. Pydantic models) that allow only expected fields; reject extras.
No input limits	Max length for messages and uploads, max tokens per request, rate limits per user / key.

Weakness	Defense
`500` **on bad input**	Validate input, handle missing objects with `404`, return generic error messages.
Verbose errors / debug mode	Disable debug in production; log details server-side only.
Client-controlled LLM settings	Backend decides model, system prompt and limits; the client sends only the user message.
Exposed docs / old versions	Restrict docs in production; keep an inventory and retire old API versions.

◆ REAL-WORLD PATTERN

In real assessments, many AI chat apps were found returning other users' conversations simply by changing a numeric conversation ID. The fix was not AI-related at all: an ownership check in the API.

4. Practice resources

OWASP API Security Top 10 owasp.org/API-Security/	**PortSwigger API testing** portswigger.net/web-security/api-testing	**MDN HTTP docs** developer.mozilla.org/docs/Web/HTTP
FastAPI tutorial fastapi.tiangolo.com/tutorial/	**curl manual** curl.se/docs/manual.html	**OWASP GenAI** genai.owasp.org

★ TAKEAWAY

Read the traffic, not the UI. Every AI feature is an HTTP request — know who controls each field, what the server checks, and what happens when input is unexpected.

PART 4

Hands-on Practical Worksheet

LAB	TIME	ENVIRONMENT
Build, explore and harden your own chat-history REST API	75–90 minutes	Python venv + FastAPI + uvicorn + curl, localhost only (127.0.0.1)

Task 1 — Build your own chat-history API

Run `pip install fastapi uvicorn` in your venv. This API has **three deliberate weaknesses** for you to find.

app.py

```
# app.py  ->  uvicorn app:app --host 127.0.0.1 --port 8000
from fastapi import FastAPI, Body, HTTPException
app = FastAPI(title="Day04 Chat History API")
DB, NEXT = {}, {"id": 1}

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/api/conversations", status_code=201)
def create(data: dict = Body(...)):
    conv = {"id": NEXT["id"], "owner": "student1", "role": "user", "messages": []}
    conv.update(data)                                   # weakness?
    DB[conv["id"]] = conv; NEXT["id"] += 1
    return conv

@app.get("/api/conversations/{cid}")
def read(cid: int):
    if cid not in DB:
        raise HTTPException(404, "not found")
    return DB[cid]

@app.post("/api/conversations/{cid}/messages")
def add_message(cid: int, data: dict = Body(...)):
    DB[cid]["messages"].append(data.get("text", ""))   # weakness? (two!)
    return {"count": len(DB[cid]["messages"])}

@app.delete("/api/conversations/{cid}", status_code=204)
def delete(cid: int):
    DB.pop(cid, None)
```

Task 2 — Explore the API documentation

Open `http://127.0.0.1:8000/docs`. Then run `curl -s .../openapi.json | jq '.paths | keys'`.

1. _____
2. _____
3. _____
4. _____

Task 3 — Fill the status code table

Send each request with `curl -i` and record the result.

Request	Status you got	Correct / expected?
`GET /health`		
`POST /api/conversations` **with** `{"title":"exam"}`		
`GET /api/conversations/1`		
`GET /api/conversations/999`		
`GET /api/conversations/abc`		
`POST /api/conversations/999/messages`		
`DELETE /api/conversations/1`		
`PUT /api/conversations/1`		

Task 4 — Read one full exchange with `curl -v`

Request / response anatomy

Request line: _____

Two request headers: _____

Status line: _____

Two response headers: _____

What does the `server` header reveal? _____

Task 5 — Find the three weaknesses

Hint: try extra fields, a missing conversation id, and a very long text.

Weakness 1 — mass assignment

Request sent: _____

Evidence (response fields): _____

OWASP API ID: _____

Impact: _____

Weakness 2 — server error on missing object

Request sent: _____

Status + body: _____

Why is this bad? _____

Weakness 3 — no input size limit

How you tested (size used): _____

Result: _____

Impact on an LLM app: _____

Task 6 — Fix with schemas, then retest

Replace `dict` bodies with strict models, return `404` for missing ids, retest every case above.

● fix sketch

```
from pydantic import BaseModel, Field, ConfigDict

class ConversationIn(BaseModel):
    model_config = ConfigDict(extra="forbid")          # unknown fields -> 422
    title: str = Field(min_length=1, max_length=100)

class MessageIn(BaseModel):
    model_config = ConfigDict(extra="forbid")
    text: str = Field(min_length=1, max_length=2000)   # size limit

# in create():      conv["title"] = data.title           (never conv.update(raw_dict))
# in add_message(): if cid not in DB: raise HTTPException(404, "not found")
```

Retest results

Mass assignment retest: _____

Missing id retest: _____

Long text retest: _____

All 3 closed? Evidence: _____

⚠ **HONESTY RULE**

If you have not run a test, write **Not tested yet** in the evidence and retest fields. Do not invent results. Real red teamers are trusted because their evidence is real.

Task 7 — Self-check quiz

Answer without looking, then check the key at the end.

Q1. Name the four main parts of an HTTP request.

Q2. What is the difference between `401` and `403`?

Q3. A chatbot's browser code sends the system prompt inside every API request. Why is that a security concern?

Q4. What is mass assignment (OWASP API3)?

Q5. Why should an API return `404`/`422` instead of `500` for bad input?

Task 8 — End-of-day checklist

- [ ] Finished all three hours (learn, build, red team).
- [ ] Ran the lab myself — not just read about it.
- [ ] Built and ran my own API on 127.0.0.1 only.
- [ ] Filled the status-code table from real curl output.
- [ ] Found, fixed and retested all three weaknesses.
- [ ] Separated assumptions from confirmed findings.
- [ ] Recorded evidence honestly (no invented results).
- [ ] Wrote my `DAY-XX.md` notes and pushed them to my notes repo / folder.
- [ ] Ticked today in the 90-Day tracker.

● **DAY-04.md (copy into your notes repo)**

```
# DAY-04.md — HTTP & REST APIs
Date:            ____ / ____ / ________
Time spent:      Learn __ min | Build __ min | Red team __ min

## 3 things I learned
1.
2.
3.

## What I built / ran (commands, code, screenshots)
-

## Findings (hypothesis -> test -> evidence -> fix -> retest)
-

## Doubts to research tomorrow
-
```

✓ ANSWER KEY — CHECK ONLY AFTER YOU TRY

Q1. Method + path (with query string), headers, and an optional body. (Plus the HTTP version on the request line.)

Q2. `401` = not authenticated (who are you?). `403` = authenticated but not allowed (I know you, but no).

Q3. Anything the client sends can be changed by the user (e.g. with Burp). The system prompt / settings should be added by the backend.

Q4. The API copies all client-supplied fields into an object, so a user can set fields they should not control, like `role` or `owner`.

Q5. `500` means unhandled errors: it hides bugs, may leak internals in debug mode, and signals weak input validation.